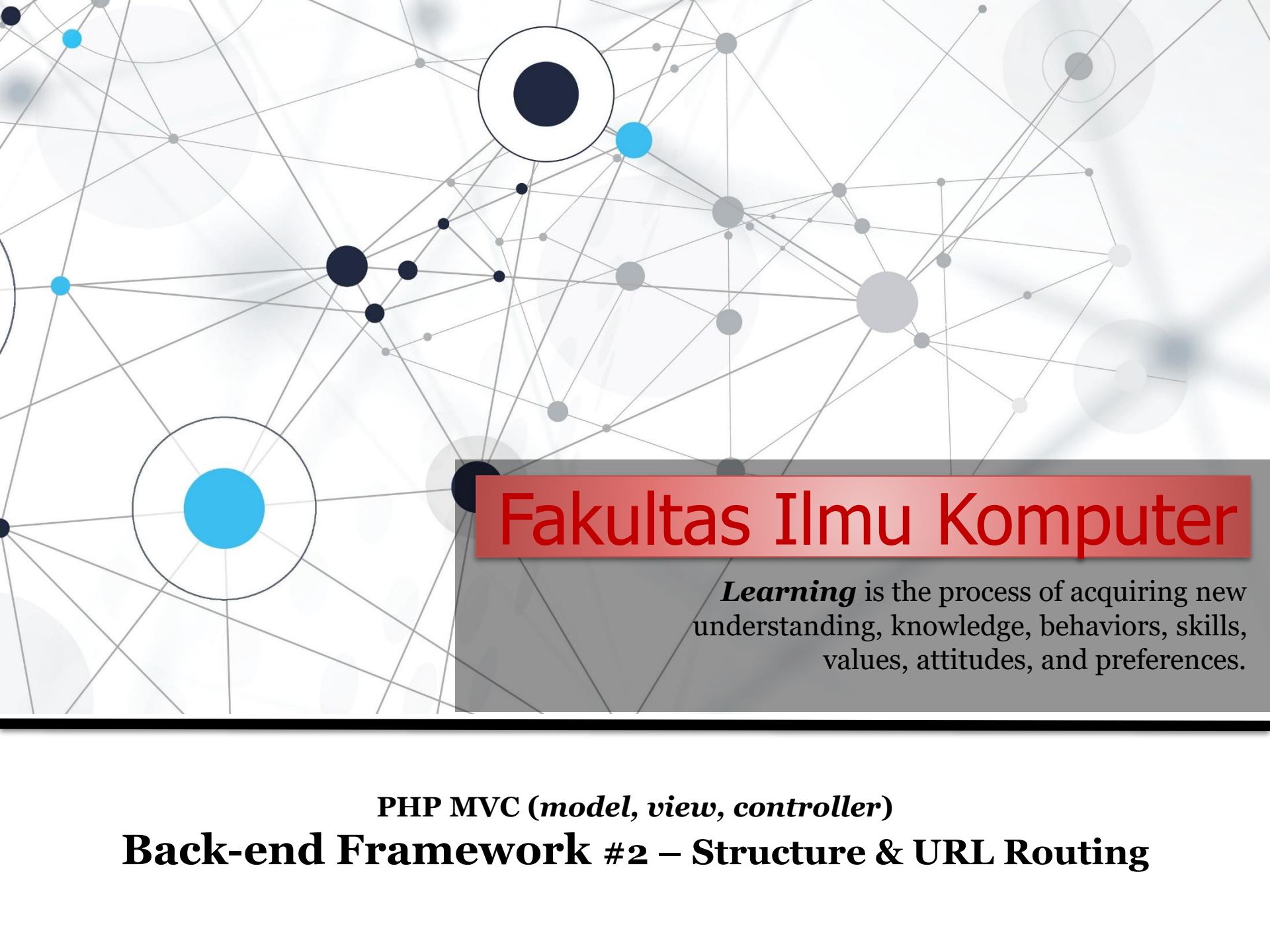


An abstract network diagram with various sized nodes (black, blue, grey) connected by thin grey lines. Some nodes are highlighted with larger circles. The background is white with faint grey circles.

Fakultas Ilmu Komputer

Learning is the process of acquiring new understanding, knowledge, behaviors, skills, values, attitudes, and preferences.

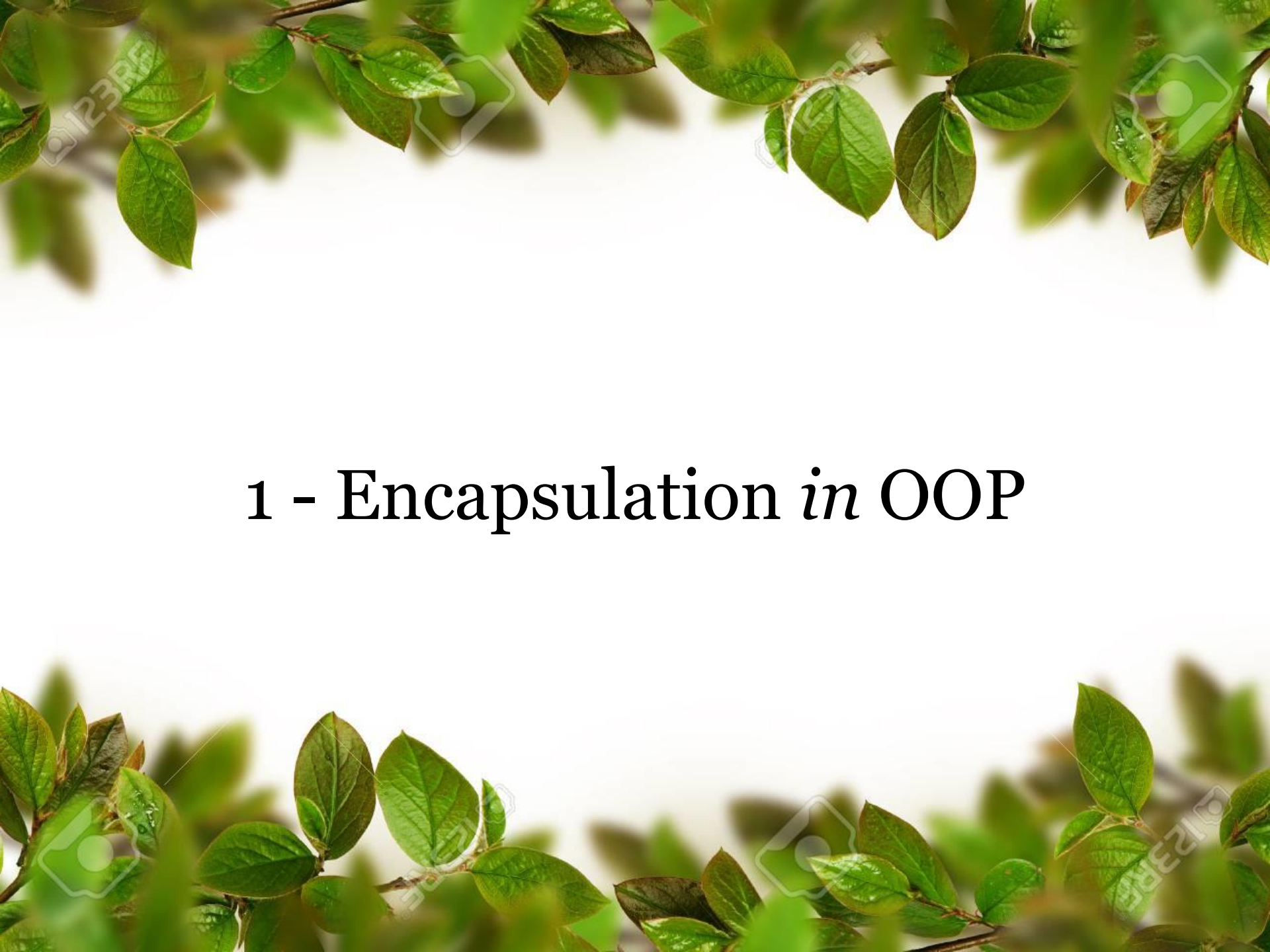
PHP MVC (*model, view, controller*)

Back-end Framework #2 – Structure & URL Routing



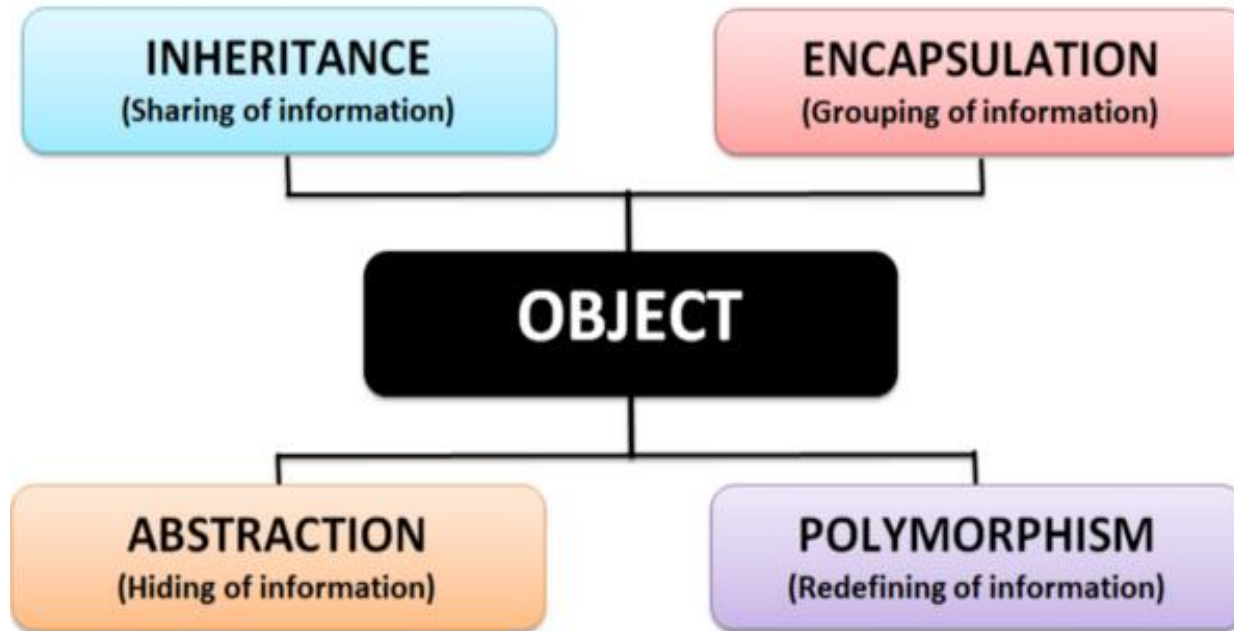
MVC File and Directory Structure

URL Parsing with Router (*URL Routing*)



1 - Encapsulation *in* OOP

Understanding *the* Concept of OOP



- ✓ Object Oriented Programming (OOP) atau pemrograman berorientasi objek adalah salah satu pendekatan paling efektif dalam menulis kode suatu program/software.
- ✓ Pada OOP, kita menulis class yang merepresentasikan suatu situasi seperti pada keadaan sebenarnya dan kita dapat membuat suatu objek berdasarkan class tersebut.

The four pillars of object-oriented programming in Python are: Abstraction; Encapsulation; Inheritance; Polymorphism.

Do you know, what is it?



What is Encapsulation?

- ❖ Encapsulation is used to **restrict access to methods and variables**. In encapsulation, code and data are wrapped together within a single unit from being modified by accident. Encapsulation isn't a **security measure in that it prevents people from messing (mengotak-atik)** with our code. It's a security measure in the sense that people **can't go directly in and change variables** without going through our proper channels.
- ❖ Encapsulation adalah salah satu konsep fundamental pada pemrograman berorientasi objek. Enkapsulasi (Encapsulation) adalah sebuah teknik dalam OOP yang mengizinkan kita untuk menyembunyikan *detail* dari sebuah *atribut* dalam sebuah *class*. Encapsulation menggambarkan suatu ide untuk membungkus data dan metode yang bekerja pada suatu data di dalam satu unit.
- ❖ Hal ini **memberikan batasan dalam mengakses** suatu variabel dan metode secara langsung. Sehingga, dapat mencegah modifikasi data yang tidak disengaja.

Ingat ini sangat penting, karena dapat mencegah perubahan yang tidak disengaja. Bukan tidak dapat di ubah, namun, variabel objek tersebut hanya dapat di ubah dengan metode objek.



Programming Technique *to* Restrict Access

Tidak Ada Sistem Yang Aman !!!

- ❖ Encapsulation prevents data modification accidentally by limiting access to variables and methods.
- ❖ We can hide the internal state of the object from the outside. This is known as information hiding.

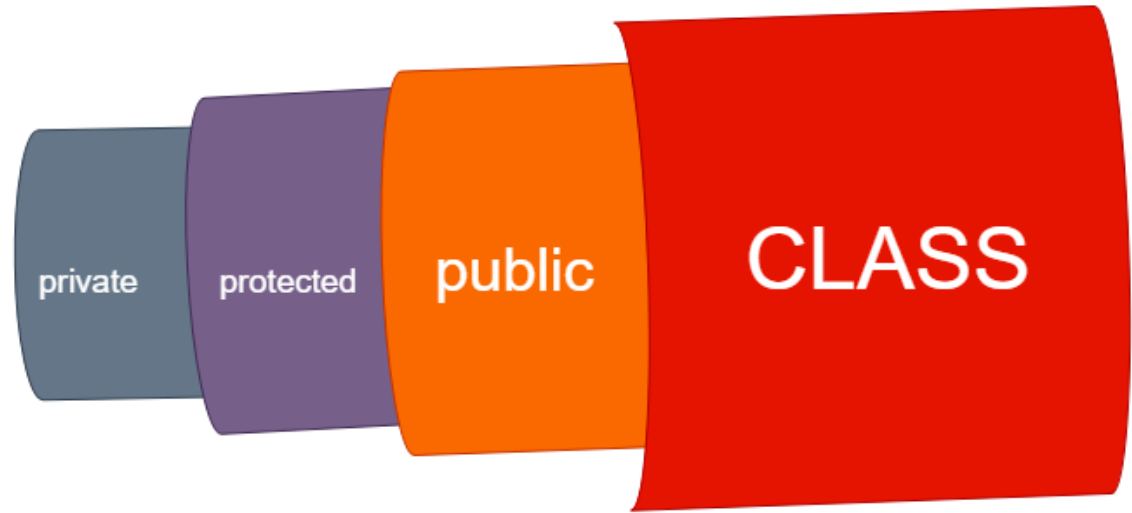


Setiap atribut dan fungsi yang telah kita definisikan belum menggunakan konsep enkapsulasi, yang mengartikan bahwa setiap atribut dan fungsi dapat diakses di luar class.

All member variables of the class are by default public.

Access Modifiers *in* Python

Encapsulation can be achieved by declaring the data members and methods of a class either as private or protected. But In Python, **we don't have direct access modifiers/keywords like public, private, and protected**. We can achieve this by using single underscore and double underscores.

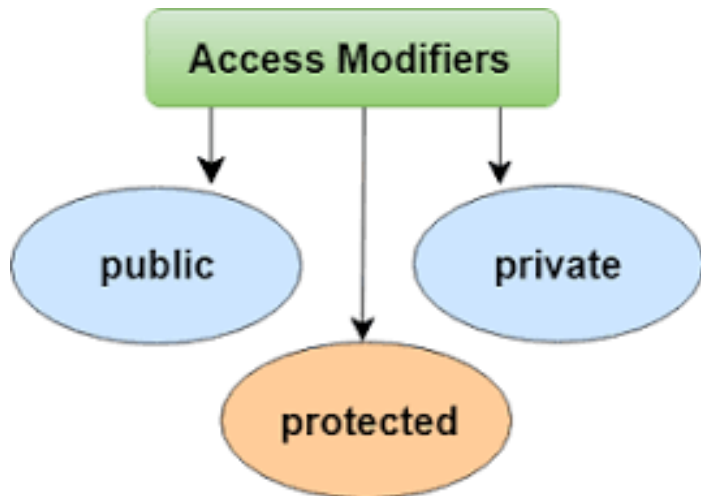


- ❖ Access modifiers limit access to the variables and methods of a class.
- ❖ Python provides three types of access modifiers private, public, and protected.

- 1) Public Member:** Accessible anywhere from outside oclass.
- 2) Private Member:** Accessible within the class
- 3) Protected Member:** Accessible within the class and its sub-classes

Access Modifiers

Access Modifiers	Same Class	Same Package	Sub Class	Other Packages
<i>Public</i>	Y	Y	Y	Y
<i>Protected</i>	Y	Y	Y	N
<i>Private</i>	Y	N	N	N



In most of the object-oriented languages access modifiers are used to limit the access to the variables and functions of a class and a package/folder.

Data hiding using access modifiers

```
class Employee:
```

```
    def __init__(self, name, salary):
```

```
        self.name = name
```

```
        self._project = project
```

```
        self.__salary = salary
```

Public Member (accessible within or outside of a class)

Protected Member (accessible within the class and it's sub-classes)

Private Member (accessible only within a class)

Data Hiding using Encapsulation

Ingat ini sangat penting, karena dapat **mencegah perubahan** yang tidak disengaja. Bukan tidak dapat di ubah, namun, variabel objek tersebut hanya dapat di ubah dengan metode objek.

Data hiding using access modifiers

```
class Employee:
```

```
    def __init__(self, name, salary):
```

```
        self.name = name
```

Public Member (accessible within or outside of a class)

```
        self._project = project
```

Protected Member (accessible within the class and it's sub-classes)

```
        self.__salary = salary
```

Private Member (accessible only within a class)

Data Hiding using Encapsulation

Dapat diperhatikan pada contoh di atas : variable *name*, *_project* dan *__salary*.

- 1) **Without underscore:** Public variable, dapat di akses dari dalam maupun luar class.
- 2) **Single underscore:** Protected variable, tidak seharusnya di akses secara langsung. Namun, tidak menghentikan Python untuk mengeksekusi perintah.
- 3) **Double underscore:** Private variable, sulit untuk di akses, namun masih memungkinkan.

Benefits of Encapsulation

- ❖ **Security:** The main advantage of using encapsulation is the **security of the data**. **Encapsulation protects an object from unauthorized access**. It allows private and protected access levels to prevent accidental data modification.
- ❖ **Data Hiding:** The *user* would not be knowing what is going on behind the scene. They would only be knowing that to modify a data member, call the setter method. To read a data member, call the getter method. What these setter and getter methods are doing is hidden from them.
- ❖ **Simplicity:** It simplifies the maintenance of the application by keeping classes separated and preventing them from tightly coupling with each other.
- ❖ **Aesthetics/estetika:** Bundling data and methods within a class makes code more readable and maintainable.

Private Access Modifier

```
1 # create class
2 class Employee:
3
4     # parameterized constructor
5     def __init__(self, name, salary):
6
7         # public data members
8         self.name = name          # public variable
9
10        # private member
11        self.__salary = salary    # private variable
12
13    # public instance methods
14    def display(self):
15        # accessing public data member
16        print("Name: ", self.name, 'Salary:', self.__salary)
17
18
19    # creating object of a class
20    unklab_emp = Employee('Semmy Wellem Taju', 2000000)
21    yzu_emp = Employee('Semmy Wellem Taju', 3000000)
22    ntu_emp = Employee('Semmy Wellem Taju', 4000000)
23    ccu_emp = Employee('Semmy Wellem Taju', 5000000)
24
25    # calling public method of the class
26    unklab_emp.display()
27    yzu_emp.display()
28    ntu_emp.display()
29    ccu_emp.display()
30
31    # accessing public data members
32    print("Name: ", unklab_emp.name, 'Salary:', unklab_emp.__salary)
33    print("Name: ", yzu_emp.name, 'Salary:', yzu_emp.__salary)
34    print("Name: ", ntu_emp.name, 'Salary:', ntu_emp.__salary)
35    print("Name: ", ccu_emp.name, 'Salary:', ccu_emp.__salary)
36
37
```

```
... # creating object of a class
... unklab_emp = Employee('Semmy Wellem Taju', 2000000)
... yzu_emp = Employee('Semmy Wellem Taju', 3000000)
... ntu_emp = Employee('Semmy Wellem Taju', 4000000)
... ccu_emp = Employee('Semmy Wellem Taju', 5000000)
...
... # calling public method of the class
... unklab_emp.display()
... yzu_emp.display()
... ntu_emp.display()
... ccu_emp.display()
Name: Semmy Wellem Taju Salary: 2000000
Name: Semmy Wellem Taju Salary: 3000000
Name: Semmy Wellem Taju Salary: 4000000
Name: Semmy Wellem Taju Salary: 5000000

In [4]: print("Name: ", unklab_emp.name, 'Salary:',
unklab_emp.__salary)
Traceback (most recent call last):

  File "C:\Users\USER\AppData\Local\Temp\ipykernel_1800\698831608.py", line 1, in <module>
    print("Name: ", unklab_emp.name, 'Salary:',
unklab_emp.__salary)

AttributeError: 'Employee' object has no attribute '__salary'

In [5]:
```

Private members are accessible only within the class, and we can't access them directly from the class objects.



Jadi, penggunaan *access modifiers* (*public*, *protected* & *private*) dan juga **URL routing** sangat penting demi tujuan keamanan.

A decorative border of green leaves and branches at the top of the slide.

2 - File & Directory Structures

A decorative border of green leaves and branches at the bottom of the slide.

Create Framework Structure

No	Directory/File	Description (<i>optional</i>)
1	<code>app</code>	app directory is the web application's <i>private directory</i> of this new framework.
2	<code>public</code>	<i>public directory</i> is to store all the public static resources like <i>html</i> , <i>css</i> , <i>image</i> and <i>js</i> files.
3	<code>core</code>	core directory is to store all files for routing and it will store the framework's core classes.
4	<code>model</code>	model directory is for all app's Model classes.
5	<code>view</code>	view directory is for all app's View classes.
6	<code>controller</code>	controller directory is for all app's Controller classes.
7	<code>css</code>	css directory is for Cascading Style Sheets (CSS) files.
8	<code>img</code>	Img directory is for images files.
9	<code>js</code>	js directory is for JavaScript files.
10	<code>index.php</code>	index.php file is the 'entry-point' file, the front controller to the users.

Additional Directories:

template directory stores the template files like *header* file and *footer* file.

config directory stores the app's configuration files.

upload directory is for uploaded files, such as uploaded images.

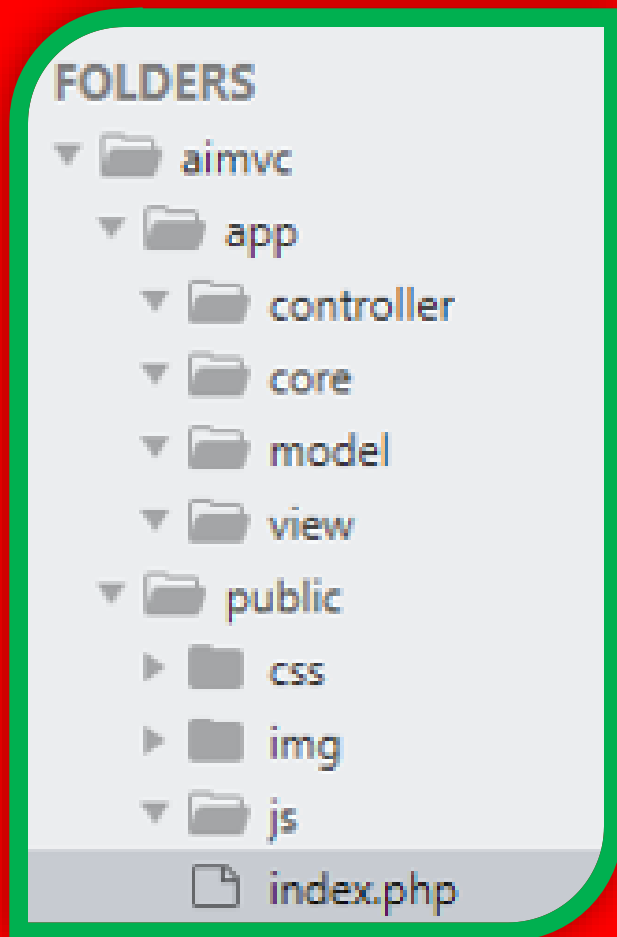
db directory is database related classes, such as database driver classes.

helper directory is help/assistant functions.

library directory is for class libraries.



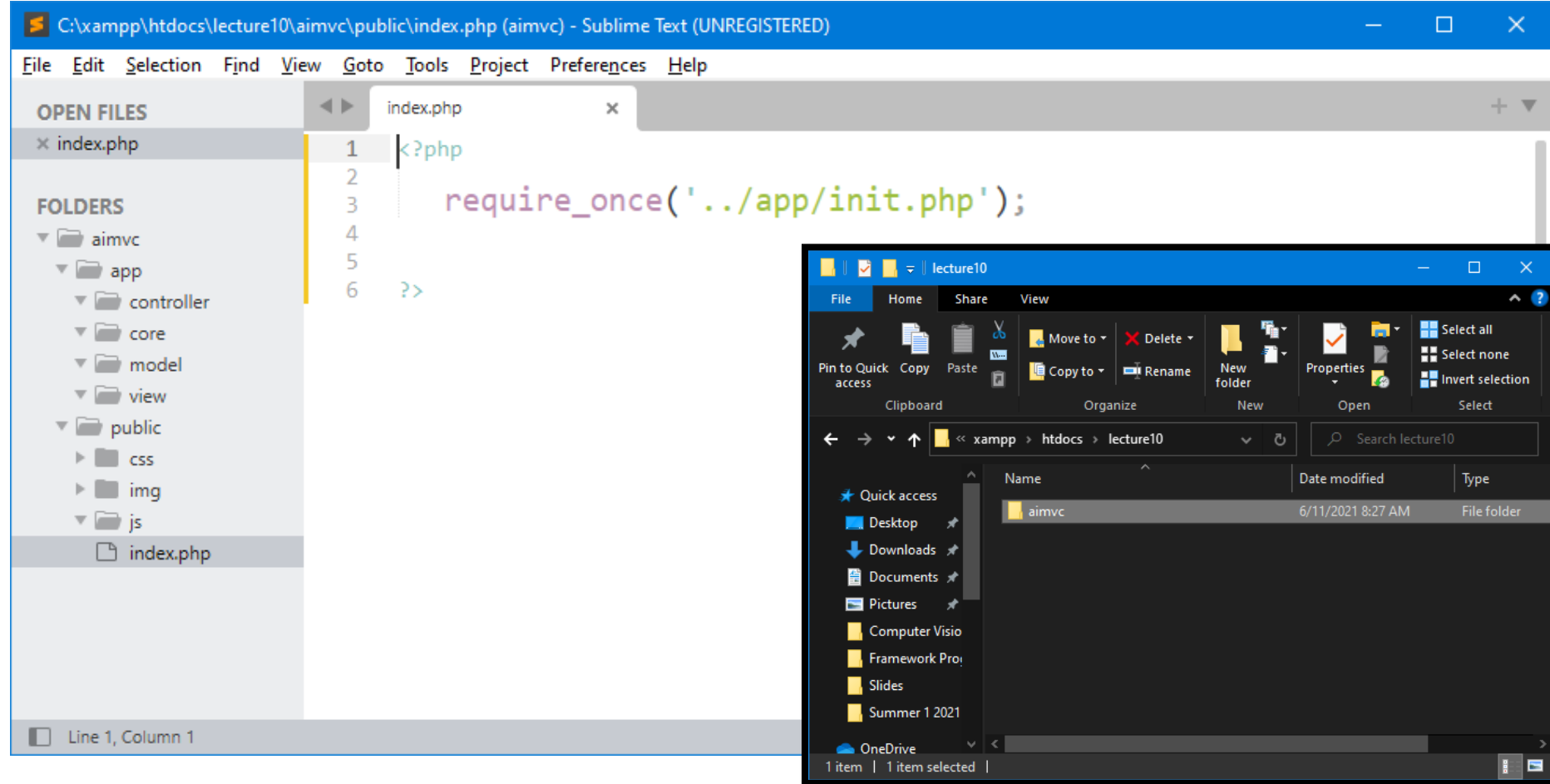
Follow the steps on the next slide!!!



As a first step,
Students di minta untuk
membuat struktur folder
seperti gambar di
samping.

Basic File & Directory Structures

Directory Path: `C:/xampp/htdocs/lecture10/aimvc/`



Main directory of our new framework:
aimvc ⇒ Artificial Intelligence Model View Controller ⇒ PHP Framework

Create The file *init.php*

init.php (*initialization*) is an optional file within the Framework file structure and located in *private directory*. This file contains the initialization of event handlers and connection of additional functions that are common for all websites.

File Path: `C:/xampp/htdocs/lecture10/aimvc/app/` (*private directory*)

```
C:\xampp\htdocs\lecture10\aimvc\app\init.php • (aimvc) - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help

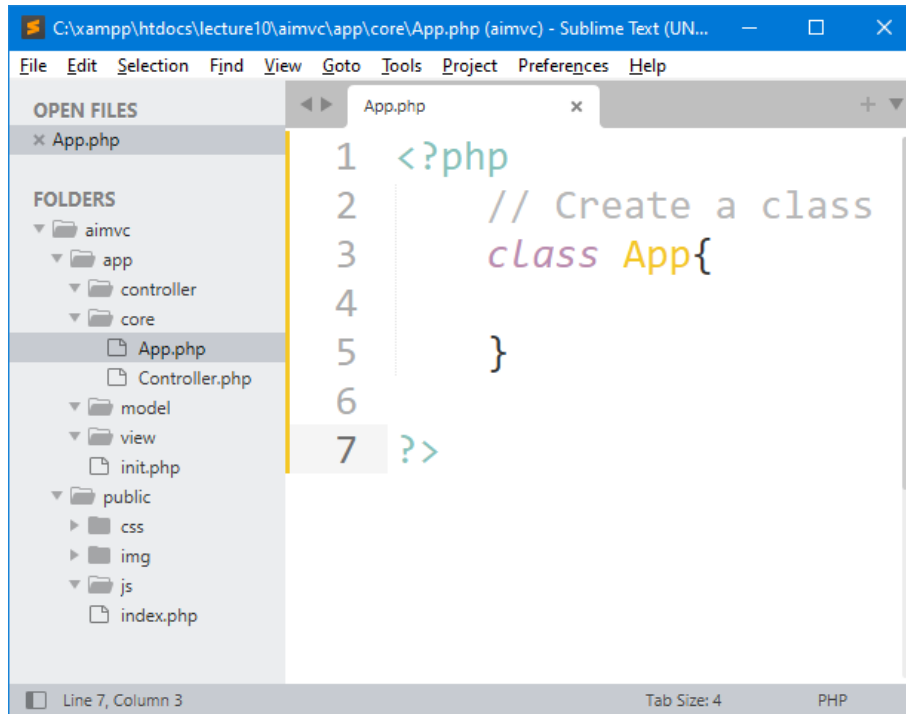
OPEN FILES
• init.php

FOLDERS
▼ aimvc
  ▼ app
    ▼ controller
    ▼ core
    ▼ model
    ▼ view
    init.php
  ▼ public
    ► css
    ► img
    ▼ js
    index.php

1 <?php
2     require_once("core/App.php");
3     require_once("core/Controller.php");
4
5 ?>
6

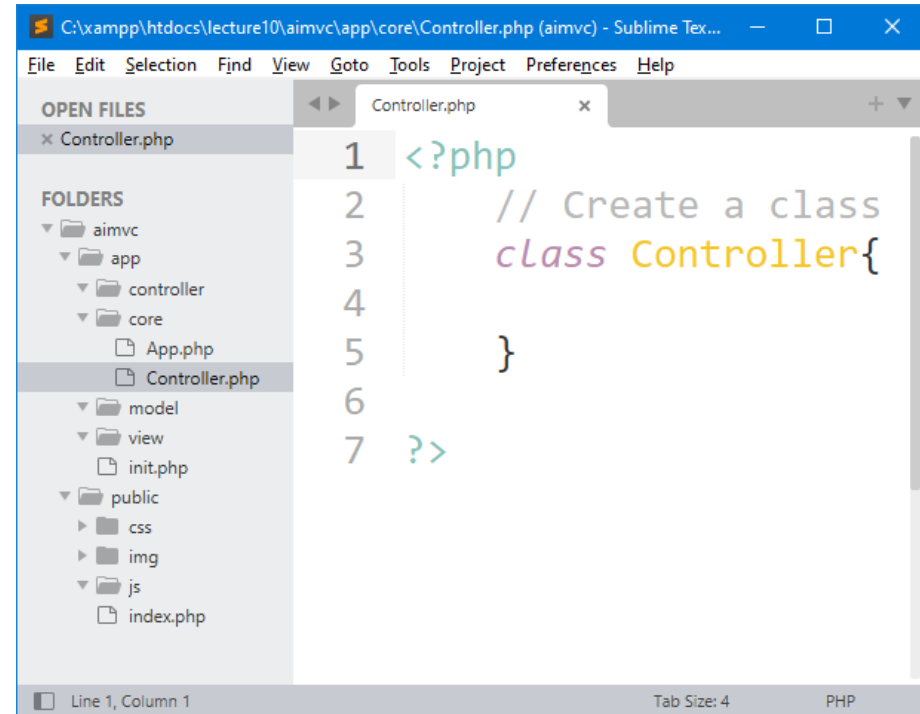
Line 1, Column 1 Tab Size: 4 PHP
```

Create Framework's Core Classes



```
1 <?php
2 // Create a class
3 class App{
4
5 }
6
7 ?>
```

The screenshot shows the Sublime Text editor with the file 'App.php' open. The left sidebar displays a file explorer with the following structure: aimvc > app > controller > core > App.php. The main editor area shows the PHP code for the 'App' class, which is currently empty except for the class declaration and closing tag. The status bar at the bottom indicates 'Line 7, Column 3'.



```
1 <?php
2 // Create a class
3 class Controller{
4
5 }
6
7 ?>
```

The screenshot shows the Sublime Text editor with the file 'Controller.php' open. The left sidebar displays a file explorer with the following structure: aimvc > app > controller > core > Controller.php. The main editor area shows the PHP code for the 'Controller' class, which is currently empty except for the class declaration and closing tag. The status bar at the bottom indicates 'Line 1, Column 1'.

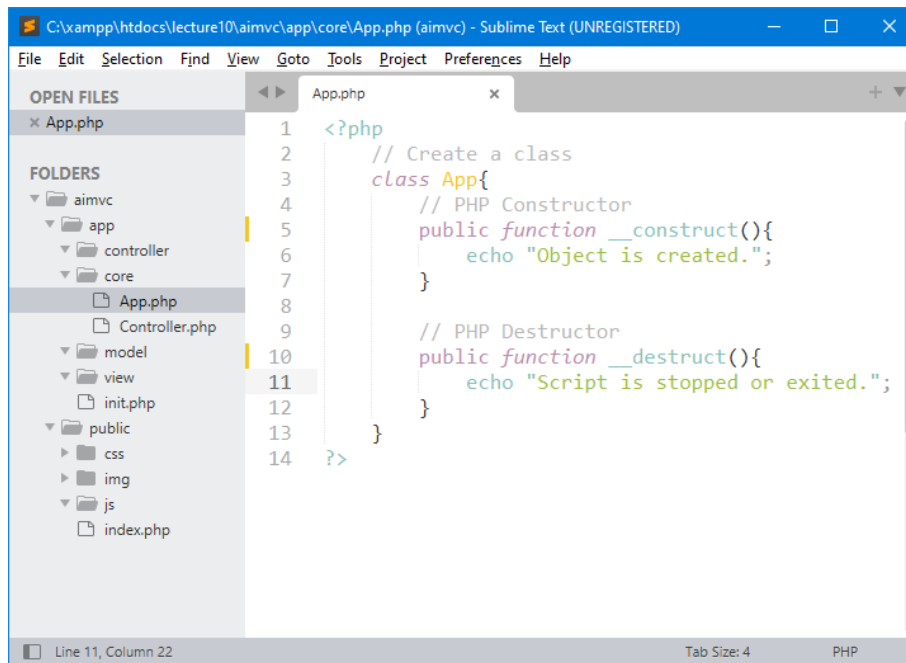
Class App (*App.php*) digunakan sebagai class utama dalam pembentukan **AIMVC** FRAMEWORK. Class Controller (*Controller.php*) digunakan sebagai *class* yang akan mengatur semua fungsi-fungsi yang akan diletakan di dalam *controller directory*.

Note:

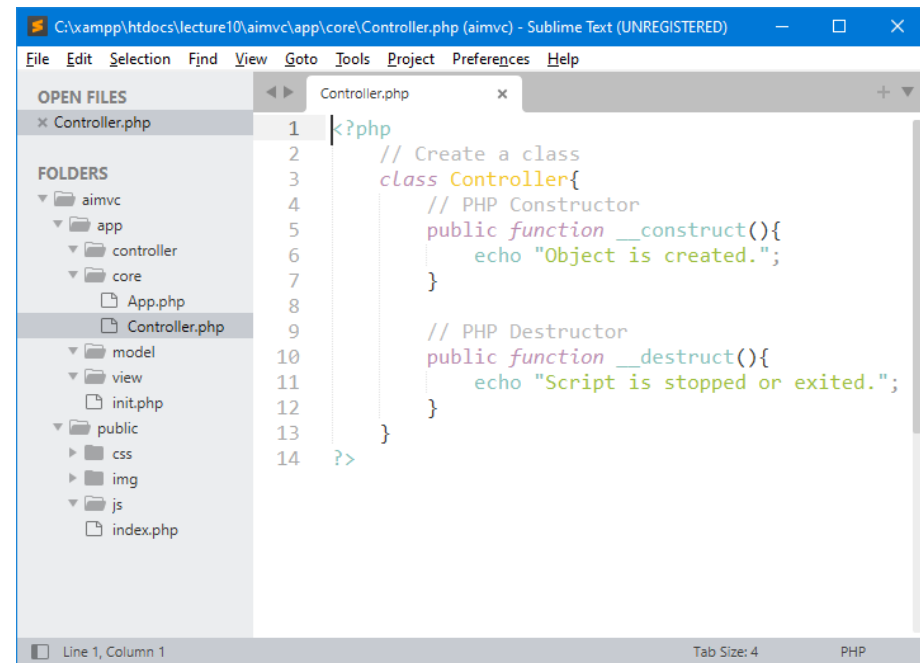
First letter as a capital letter (*uppercase letter*) normally means a PHP class.

Constructors & Destructors

Tambahkan fungsi **__construct** dan **__destruct** di Class App (**App.php**) dan Class Controller (**Controller.php**), ini berfungsi agar supaya pada saat proses instansiasi atau proses pembuatan instance/objek dari class App (**App.php**) dan class Controller (**Controller.php**) selesai, maka sudah bisa langsung di *display* contoh output-nya.

A screenshot of the Sublime Text editor window titled 'C:\xampp\htdocs\lecture10\aimvc\app\core\App.php (aimvc) - Sublime Text (UNREGISTERED)'. The 'OPEN FILES' panel on the left shows 'App.php' selected. The 'FOLDERS' panel shows a directory structure with 'App.php' and 'Controller.php' under the 'core' folder. The main editor area displays the following PHP code:

```
1 <?php
2 // Create a class
3 class App{
4     // PHP Constructor
5     public function __construct(){
6         echo "Object is created.";
7     }
8
9     // PHP Destructor
10    public function __destruct(){
11        echo "Script is stopped or exited.";
12    }
13 }
14 ?>
```

The status bar at the bottom indicates 'Line 11, Column 22' and 'PHP'.A screenshot of the Sublime Text editor window titled 'C:\xampp\htdocs\lecture10\aimvc\app\core\Controller.php (aimvc) - Sublime Text (UNREGISTERED)'. The 'OPEN FILES' panel on the left shows 'Controller.php' selected. The 'FOLDERS' panel shows the same directory structure as the previous screenshot. The main editor area displays the following PHP code:

```
1 <?php
2 // Create a class
3 class Controller{
4     // PHP Constructor
5     public function __construct(){
6         echo "Object is created.";
7     }
8
9     // PHP Destructor
10    public function __destruct(){
11        echo "Script is stopped or exited.";
12    }
13 }
14 ?>
```

The status bar at the bottom indicates 'Line 1, Column 1' and 'PHP'.

PHP - The **__construct** Function:

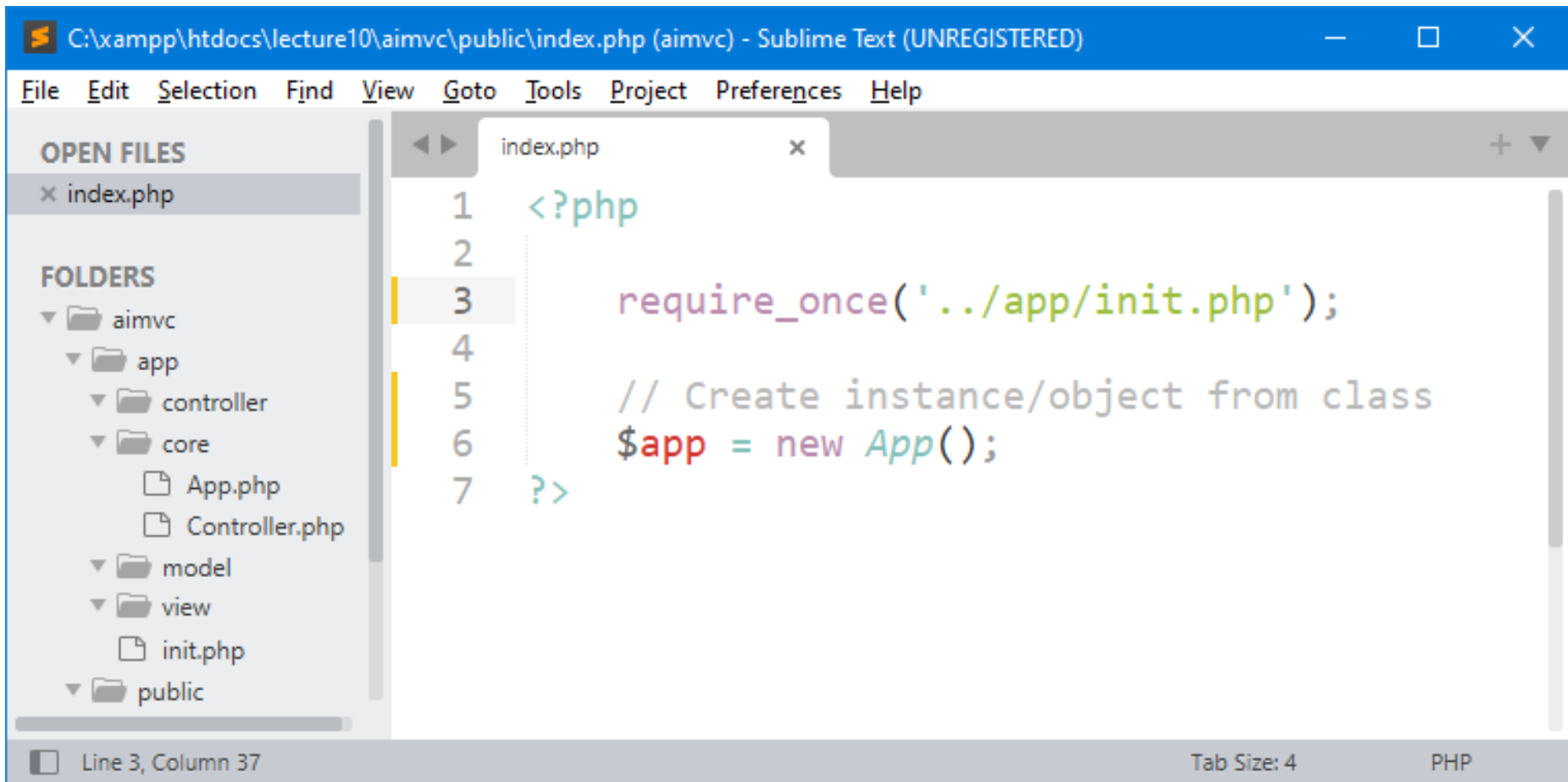
A constructor allows you to initialize an object's properties upon creation of the object.

PHP - The **__destruct** Function:

A destructor is called when the object is destructed or the script is stopped or exited.

Instansiasi Class (Object Class) App

Instansiasi **class App** (*App.php*) dari file **index.php** adalah pembuatan instance/objek dari *class* App, dimana kita akan coba menampilkan/view sesuatu dari *class* tersebut. Perhatikan di dalam *constructor* dari class App sudah terdapat *statement echo* (*which is used to display the output*) dimana akan digunakan untuk menampilkan *output* sederhana.



```
C:\xampp\htdocs\lecture10\aimvc\public\index.php (aimvc) - Sublime Text (UNREGISTERED)

File Edit Selection Find View Goto Tools Project Preferences Help

OPEN FILES
x index.php

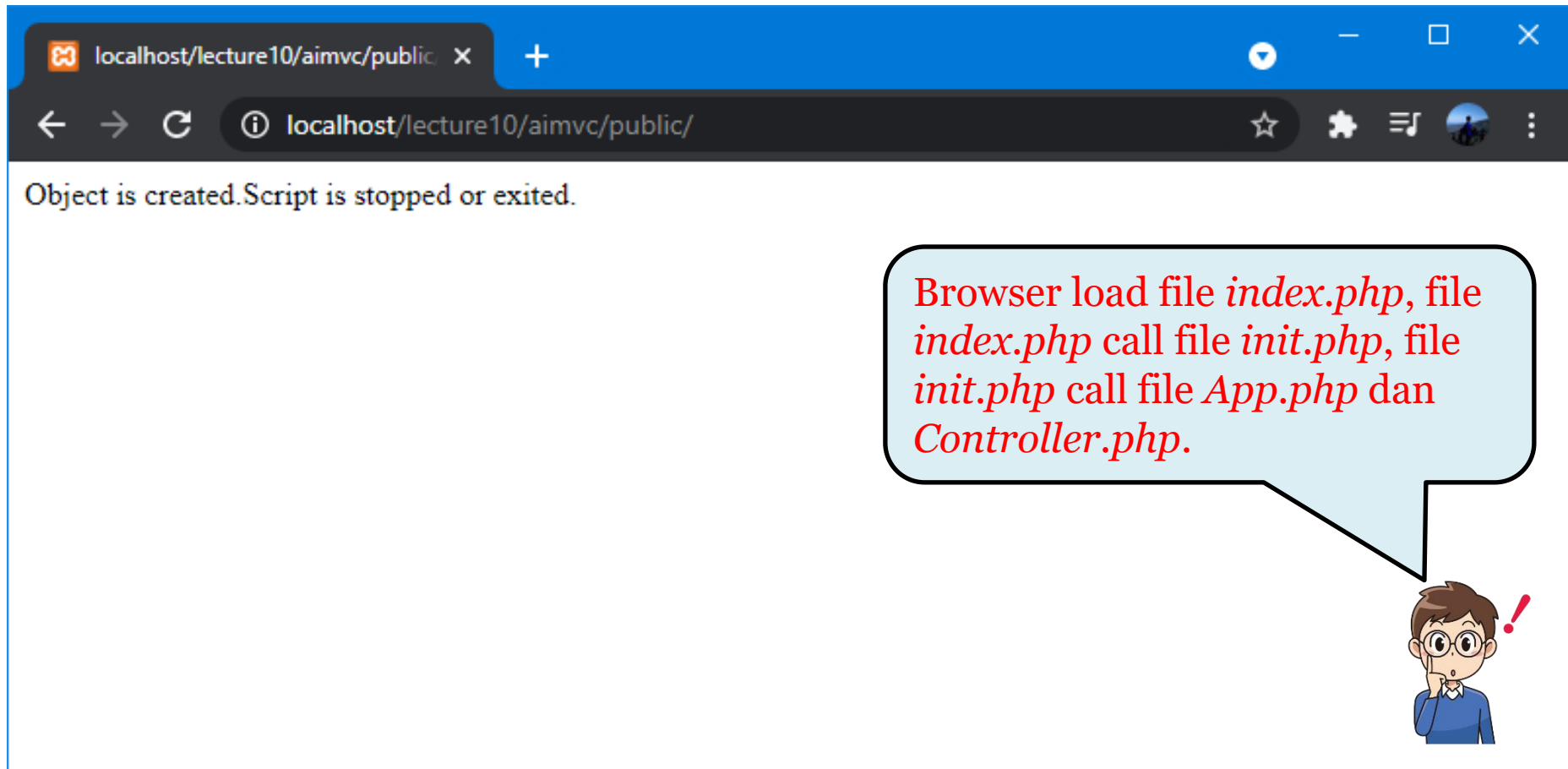
FOLDERS
aimvc
  app
    controller
    core
      App.php
      Controller.php
    model
    view
      init.php
  public

1 <?php
2
3     require_once('../app/init.php');
4
5     // Create instance/object from class
6     $app = new App();
7 ?>
```

Line 3, Column 37 Tab Size: 4 PHP

Test *your* Framework's Structure

Uniform Resource Locator (URL): <http://localhost/lecture10/aimvc/public/>



Pada saat jalankan (*run*) PHP Files di *browser*, maka *output* yang diharapkan (*giving the expected output*) yaitu seperti diatas.

A decorative border of green leaves and branches at the top of the slide.

3 - Penerapan *Routing* (*route*)

[*URL Parsing with Router or URL Routing*]

A decorative border of green leaves and branches at the bottom of the slide.

Istilah/Term Routing

- ❑ **Routing** adalah proses dimana suatu *item* dapat sampai ke tujuan dari satu lokasi ke lokasi lain. Dimana, *item* yang dimaksud adalah halaman aplikasi website. *Developer* dapat menentukan sendiri *page* yang akan muncul pada saat dikunjungi oleh *users*.
- ❑ **Routing** is the process of selecting a path for traffic in a network or between or across multiple networks. Broadly, routing is performed in many types of networks, including circuit-switched networks, such as the public switched telephone network (PSTN), and computer networks, such as the Internet. -

Wikipedia

Contoh penggunaan Routing:

Pada saat *user* mengunjungi halaman *dashboard*, maka *kita* dapat menentukan tampilan apa yang akan muncul, apakah itu hanya berupa tulisan (*text*), berupa halaman *controller*, berupa halaman *view*, maupun halaman *error*.

Route dapat handle semua perintah yang telah dideklarasikan atau bahkan **route** bisa dianalogikan sebagai peta petunjuk bagaimana alur navigasi aplikasi yang sedang dibangun atau juga bisa disebut *Router* merupakan suatu rute/jalan/trayek.

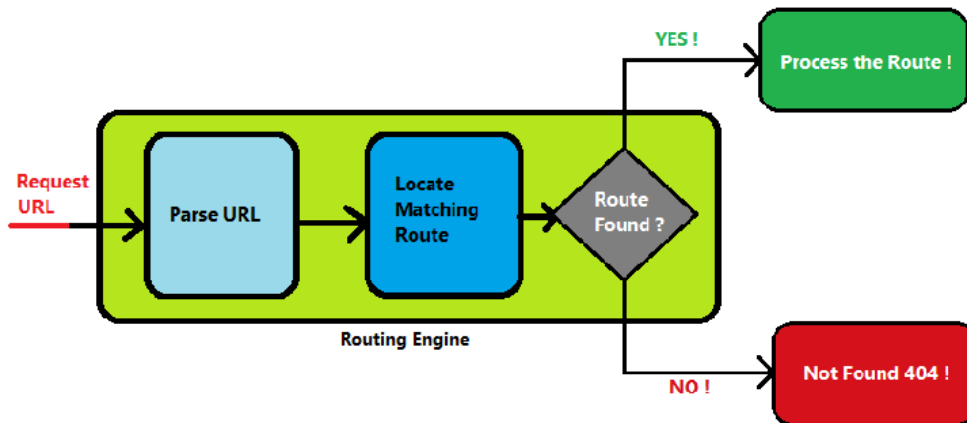
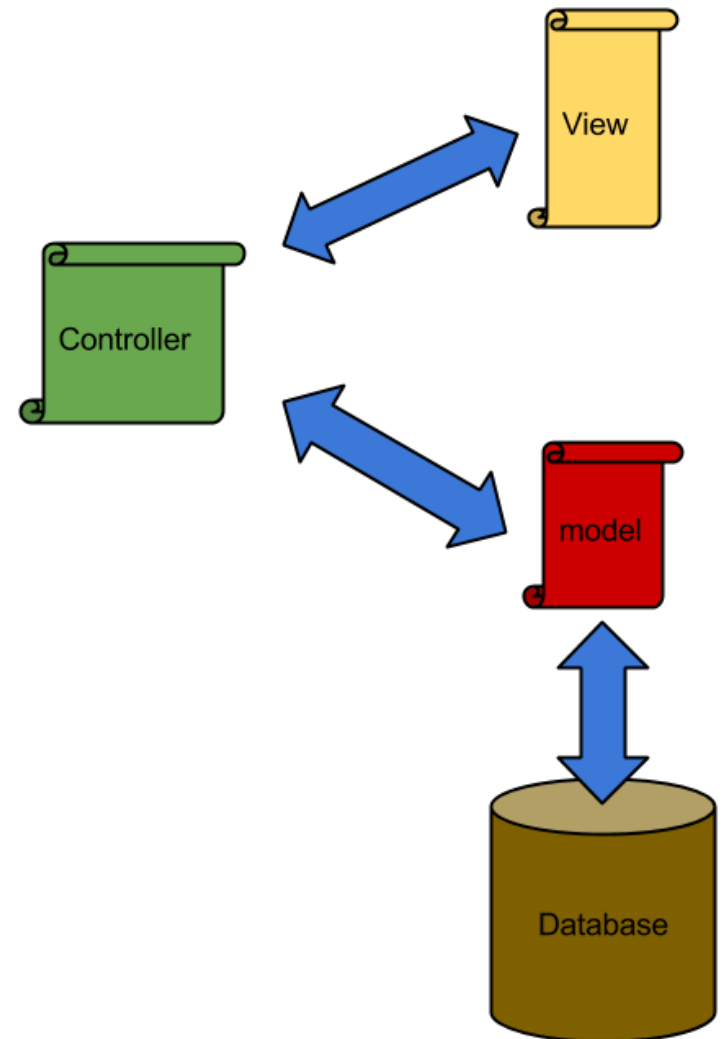
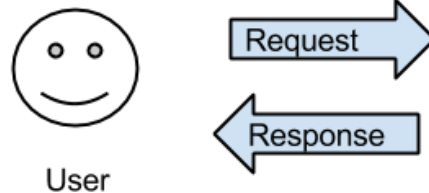


Framework Routing

Server Side

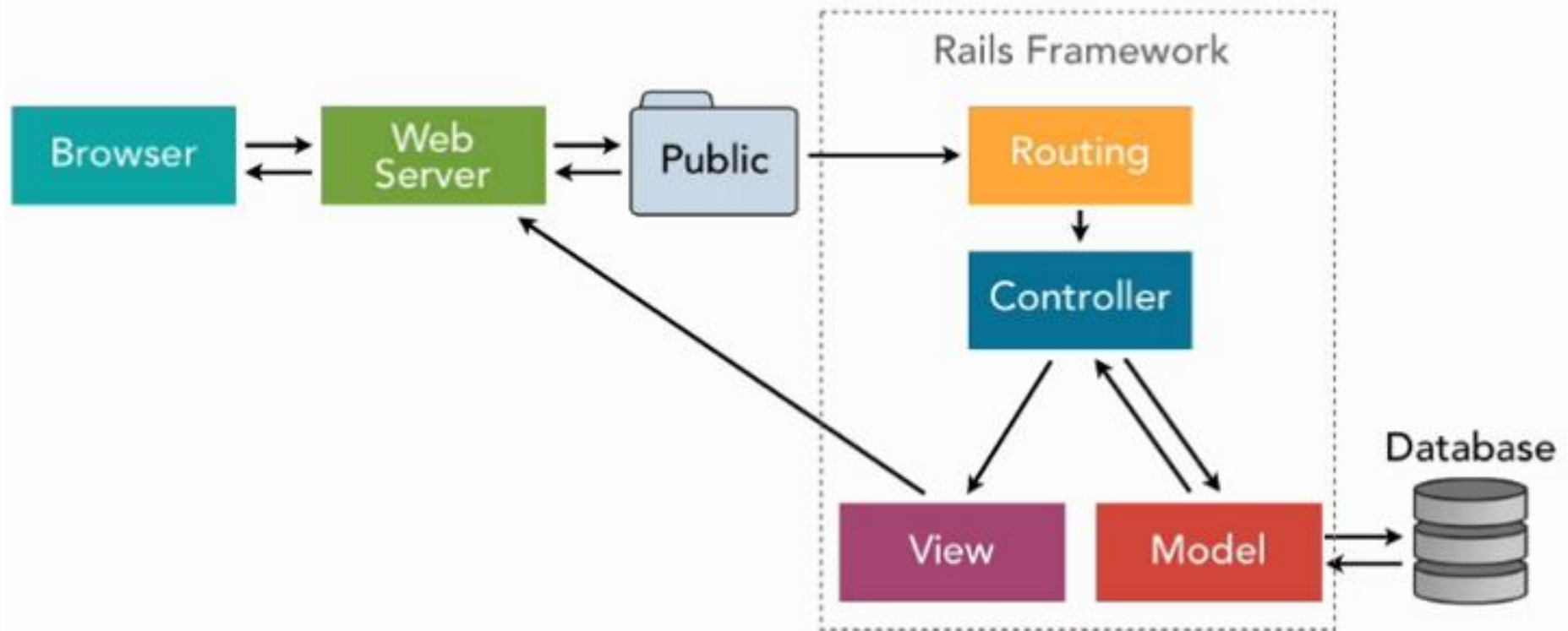
Client Side

Pada saat *user* mengunjungi halaman *home*, maka kita dapat menentukan tampilan apa yang akan muncul, apakah itu hanya berupa tulisan (*text*), berupa halaman *controller*, berupa halaman *view*, maupun halaman *error*.



Rails Architecture

Ruby on Rails, or **Rails**, is a server-side web application framework written in Ruby under the MIT License. Rails is a ***model–view–controller*** (MVC) framework, providing default structures for a database, a web service, and web pages.



We now change the way to handle URL Parameters as an input to the server

URL (Uniform Resource Locator) isn't just a plain address. It contains extra information added to the end to help the web page perform something specific.



URL Routing

Typically there is a one-to-one relationship (*hubungan satu-ke-satu*) between a URL string and its corresponding controller class/method. The segments in a URL (*bagian dalam URL*) normally follow this pattern:

`http://www.example.com/class/method/parameter1/parameter2/`

What is a Controller?

A Controller is simply a class file that is named in a way that can be associated with a URI.

Class merupakan *controller class* yang sudah dibuat dan biasanya segmen kedua (**method**) dari URL disiapkan untuk nama *metode*.

Attendance

https://unkpresent.site/unklab/public/students/attendance/16555

Universitas Klabat

Search

Sign out

Semmy Wellem Taju

GENERAL MENU

- Home
- Informations
- Classroom Topics
- Logout

SHORTCUT LECTURER

- Active Classes
- Archived Classes
- Attendance
- Magic Cut

Class Attendance Checklist

Course: [IF3262] Back-End Web Development - D

New + Summary Back →

	Course Topic	Schedule	Present/Total	Radius	Actions
☒	Lecture9 - Class Activity #1 MVC Architectural Pattern Taju, Semmy Wellem semmy@unklab.ac.id	Wednesday 07 February 2024 14.30 - 16.00	25/27 Students	100 Meter	Options
☒	Lecture8 - Framework #1 [MVC Architectural Pattern] Taju, Semmy Wellem semmy@unklab.ac.id	Monday 05 February 2024 14.30 - 16.00	24/27 Students	100 Meter	Options
☒	Lecture7 - Ajax for PHP Web Applications Taju, Semmy Wellem semmy@unklab.ac.id	Wednesday 31 January 2024 14.30 - 16.00	25/27 Students	100 Meter	Options
☒	Lecture6 - Magic Constants in PHP Taju, Semmy Wellem semmy@unklab.ac.id	Monday 29 January 2024 14.30 - 16.00	25/27 Students	100 Meter	Options
☒	Lecture5 - Cookies, Session & HTAccess Taju, Semmy Wellem semmy@unklab.ac.id	Wednesday 24 January 2024 14.30 - 16.00	24/27 Students	80 Meter	Options
☒	Lecture4 - Classes and Objects Taju, Semmy Wellem semmy@unklab.ac.id	Monday 22 January 2024 14.30 - 16.00	26/27 Students	100 Meter	Options

https://unkpresent.site/**unklab**/**public**/**students**/**attendance**/16555

Menurut pendapat anda, kira-kira mana saja yang termasuk dalam URL parameter request?

Universitas Klabat

Search

Sign out

Semmy Wellem Taju ▾

GENERAL MENU

Home

Informations

Classroom Topics

Logout

SHORTCUT LECTURER

Active Classes

Archived Classes

Attendance

Magic Cut

Besouw, Christian Juan Frederik

Course: [IF4296] DevOps Engineering - A

Active Classes

Back →

No.	Course Topic	Day/Date	Time	Status
1	Lecture2 - Introduction to DevOps Engineering  Taju, Semmy Wellem	Tuesday 16 January 2024	14.30 - 16.00	Late
2	Lecture3 - Client Side Programming (HTML, CSS & JavaScript)  Taju, Semmy Wellem	Thursday 18 January 2024	14.30 - 16.00	Absen
3	Lecture4 - Server Side Programming (PHP, Python, Node.JS & Ruby)  Taju, Semmy Wellem	Tuesday 23 January 2024	14.30 - 16.00	Late
4	Lecture5 - (Class Practical #1) Development & Monitoring Windows Apache Error  Taju, Semmy Wellem	Thursday 25 January 2024	14.30 - 16.00	Late
5	Lecture7 - Serverless Computing, Cloud Computing & Cloud VPS - PART 1  Taju, Semmy Wellem	Thursday 01 February 2024	14.30 - 16.00	Absen
6	Lecture8 - Serverless Computing, Cloud Computing & Cloud VPS - PART 2  Taju, Semmy Wellem	Tuesday 06 February 2024	14.30 - 16.00	Absen

<https://unkpresent.site/unklab/public/students/itemize/16718/QmVzb3V3LCBDbHJpc3RpYW4gSnVhbiBGcmVhZjZlMmFjLmlk>

Menurut pendapat anda, kira-kira ada berapa banyak parameter request yang digunakan untuk URL di atas?

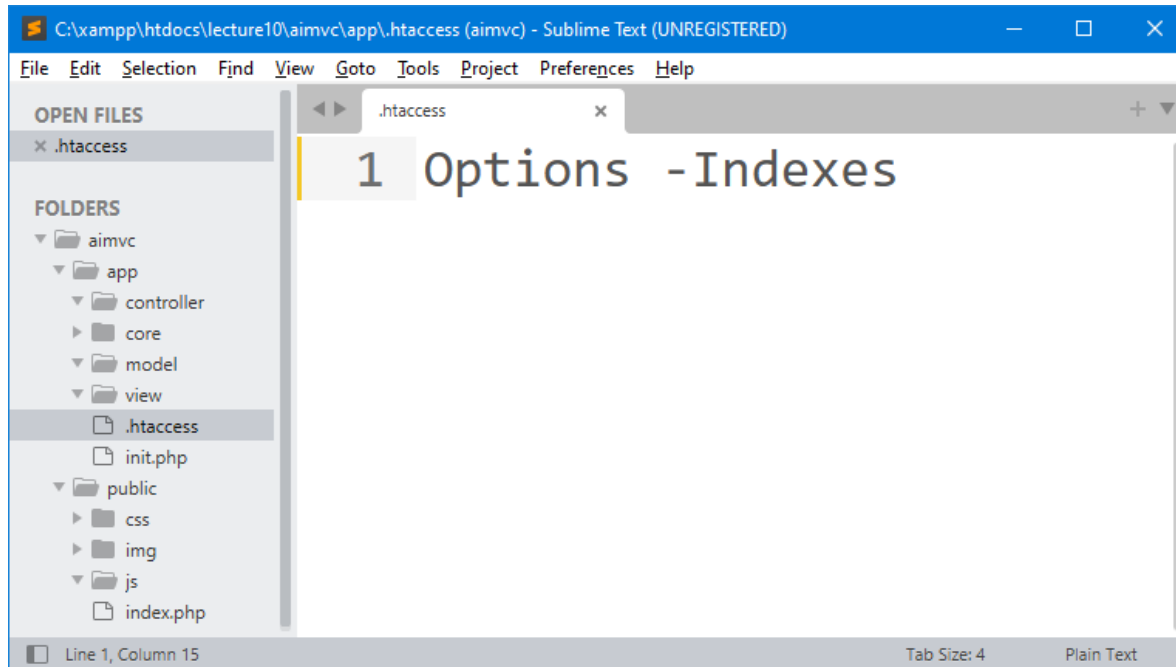


Follow the steps on the next slide!!!

**Setting *your* own
routing rules**

1 - Restrict Users Access (*private*)

Block/restrict access permission to ***private*** directory (*app directory*):

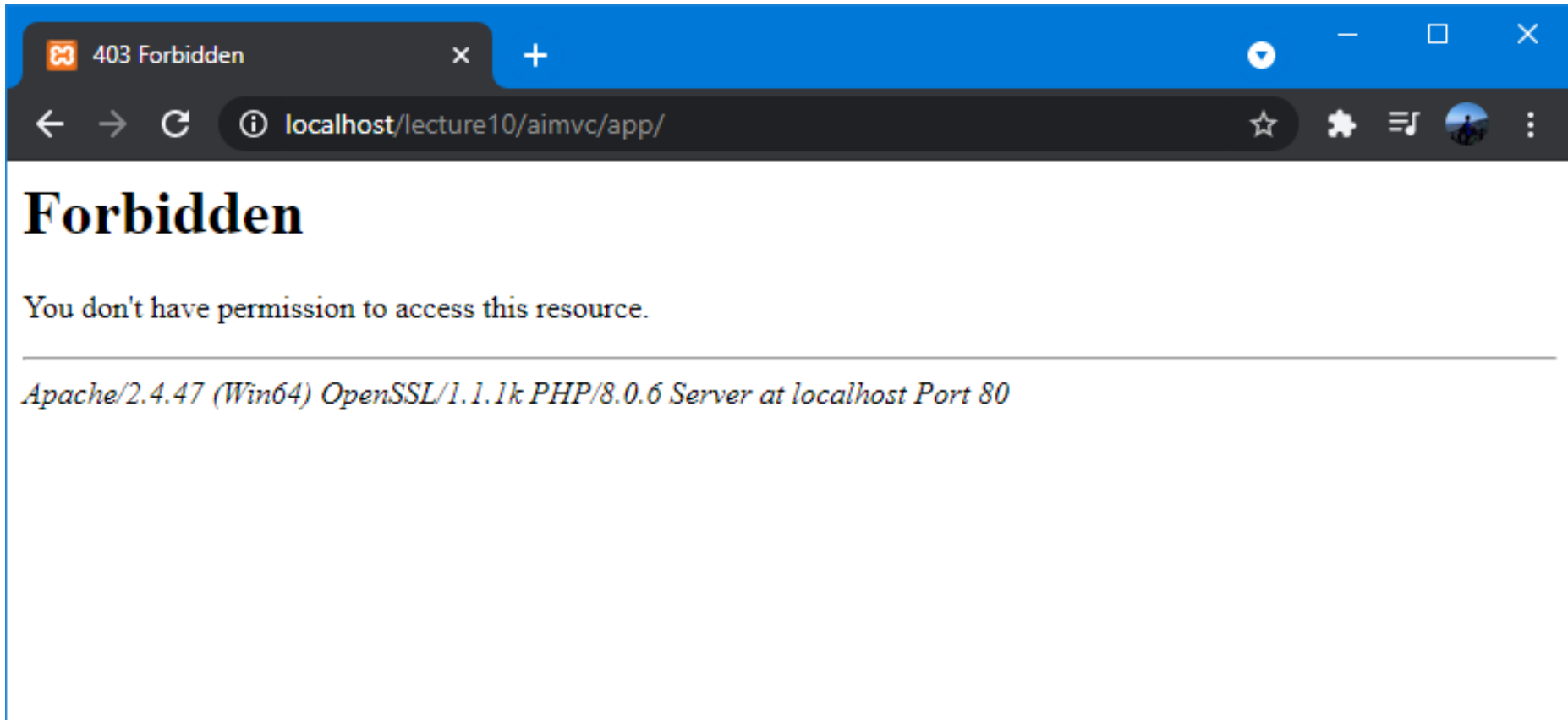


Create file **.htaccess** di dalam folder app. “**Options –Indexes**” artinya jika ada *users* membuka folder app dan folder-folder yang lain di dalamnya maka **jangan tampilkan isi folder-nya**, selama di dalam *folder* tersebut tidak ada file *index*. Dalam arti user tidak dapat diberi izin.

- ❖ File **.htaccess** adalah cara untuk mengkonfigurasi rincian website kita tanpa perlu mengubah *file konfigurasi server*.
- ❖ **Options -Indexes** hanya melarang directory listing (*list file muncul di browser*). **TIDAK** melarang akses langsung ke file di dalamnya.

1 - Restrict Users Access (*private*)

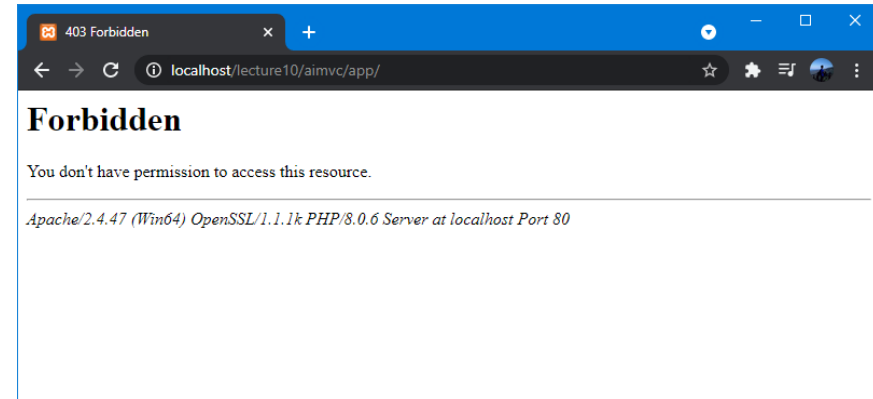
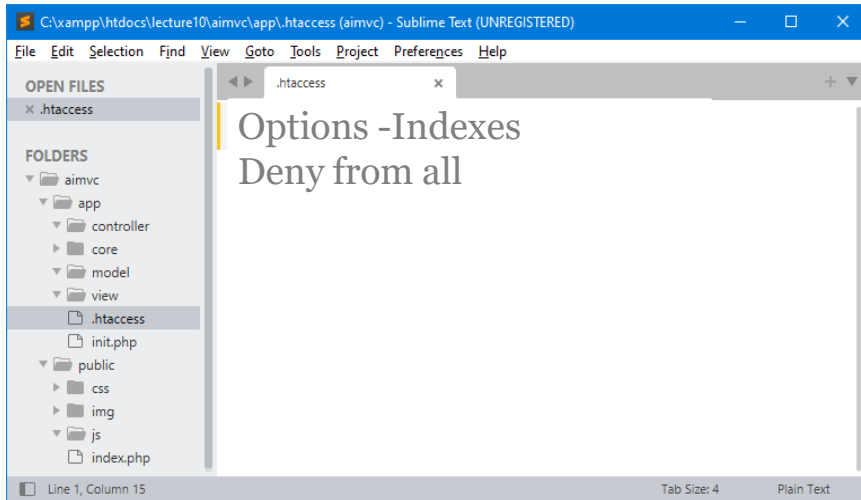
Test apakah *users* masih diberi *access* permission ke ***private*** directory (*app directory*) dan folder-folder yang lain di dalamnya? Jika iya, maka dapat mengaktifkan *.htaccess* file pada *httpd.conf* file.



Test dibuka juga directory <http://localhost/lecture10/aimvc/app/view/>

1 – Block Akses dari luar (browser)

Remember, **Options -Indexes** hanya melarang directory listing (list file muncul di browser). **TIDAK** melarang akses langsung ke file di dalamnya.



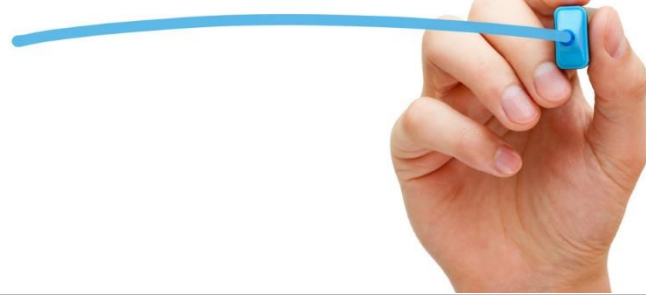
Hanya berlaku untuk **request HTTP** dari luar (browser / curl).

```
1  # Jangan tampilkan daftar isi folder
2  Options -Indexes
3
4  # Tolak semua akses dari luar
5  <Files *>
6      Order allow,deny
7      Deny from all
8  </Files>
9
```

Jadi :

- ❖ **Options -Indexes** → cegah list isi folder dari sisi client/browser.
- ❖ **Deny from all** → memblokir semua request HTTP dari luar ke folder/file tempat .htaccess itu berada.
- ❖ Gabungan ini artinya: Folder itu **benar-benar private**, hanya bisa diakses lewat include/require dari script PHP lain di server.

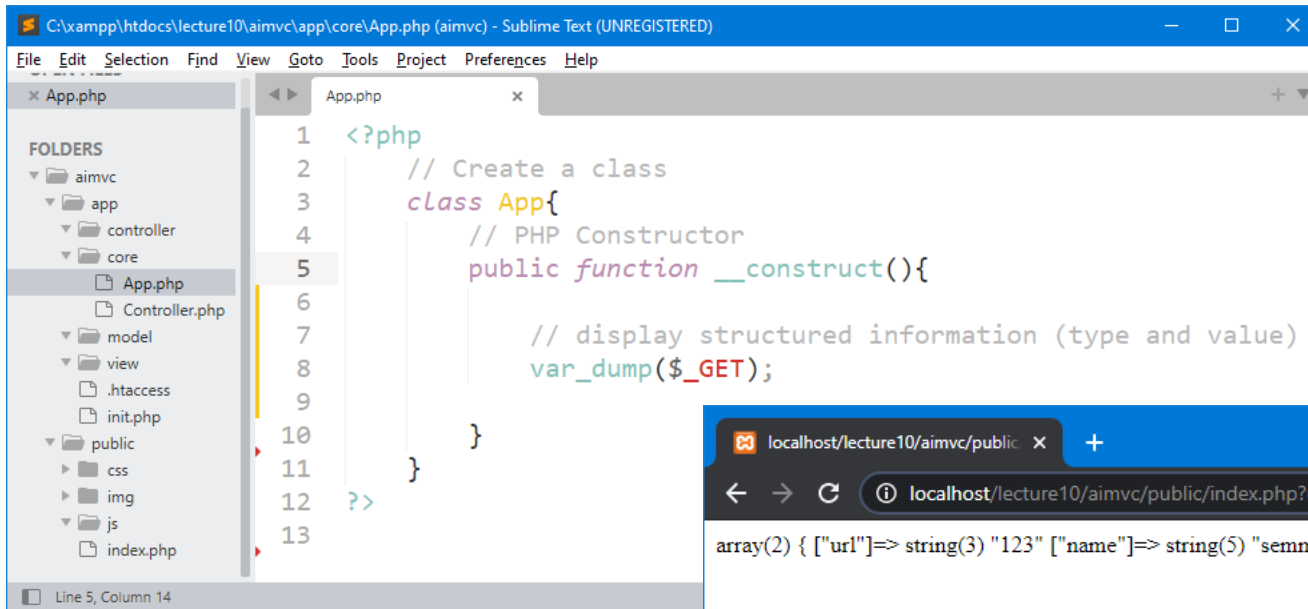
BEST PRACTiCE



Best practice:

Folder `config` **tidak bisa diakses langsung lewat URL** (dibatasi `Deny from all` atau ditempatkan di luar `DocumentRoot`).

2 - Test Request URL Public Directory

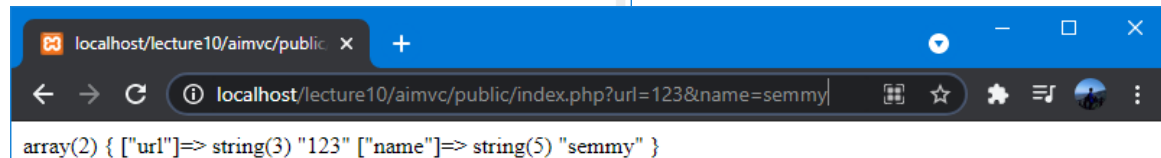


The screenshot shows a Sublime Text editor window titled "C:\xampp\htdocs\lecture10\aimvc\app\core\App.php (aimvc) - Sublime Text (UNREGISTERED)". The left sidebar displays a file explorer with the following structure:

- aimvc
 - app
 - controller
 - core
 - App.php**
 - Controller.php
 - model
 - view
 - .htaccess
 - init.php
 - public
 - css
 - img
 - js
 - index.php

The main editor area shows the content of App.php:

```
1 <?php
2 // Create a class
3 class App{
4     // PHP Constructor
5     public function __construct(){
6
7         // display structured information (type and value)
8         var_dump($_GET);
9
10    }
11 }
12 ?>
13
```



The screenshot shows a web browser window with the address bar displaying "localhost/lecture10/aimvc/public/index.php?url=123&name=semmy". The browser's developer tools or output area shows the following PHP output:

```
array(2) { ["url"]=> string(3) "123" ["name"]=> string(5) "semmy" }
```

Request URL:

<http://localhost/lecture10/aimvc/public/index.php?url=123&name=semmy>

Jika dilihat URL yang di *request* tidak teratur, oleh sebab itu kita akan mencoba Parsing URL (*mengurai URL*) dan membuatnya lebih rapi sesuai dengan *standard framework* yang akan di *design*.

Apply PHP isset() Function in URL

We used *isset function* to check whether a variable (*get url*) is empty and also check whether the variable (*get url*) is set/declared before setting our own routing rules.

Example isset function:

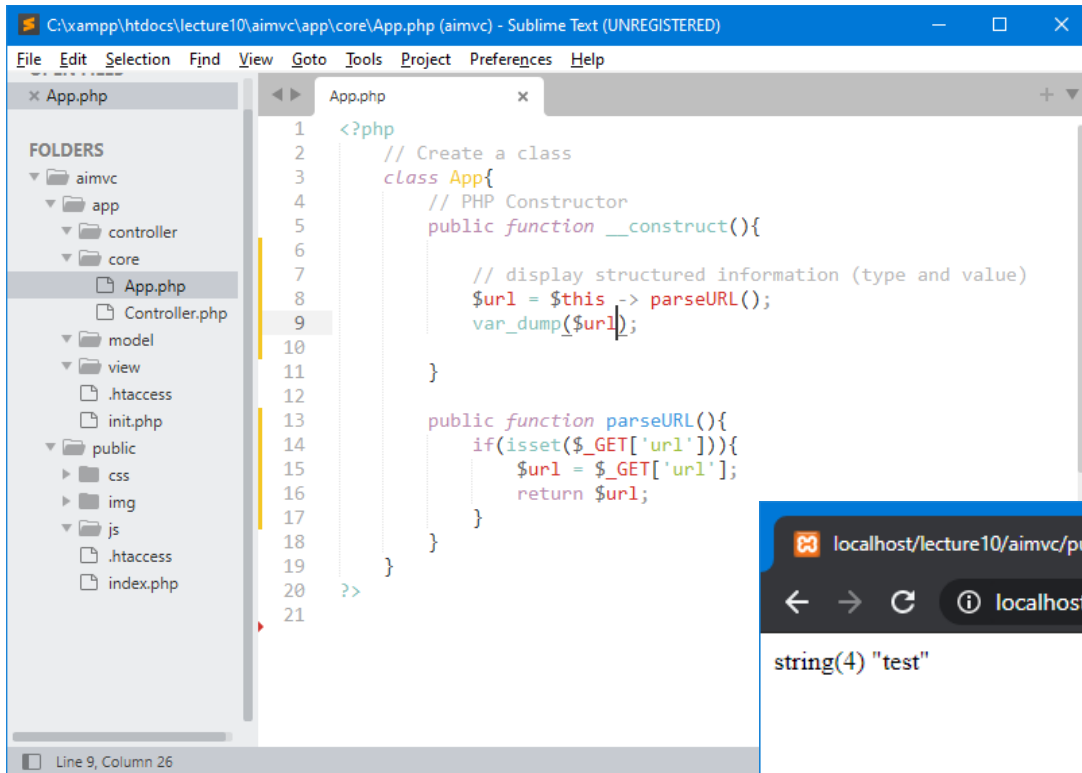
```
<!DOCTYPE html>
<html>
<body>
  <?php
    $a = 0;
    // True because $a is set
    if (isset($a)) {
      echo "Variable 'a' is set.<br>";
    }

    $b = null;
    // False because $b is NULL
    if (isset($b)) {
      echo "Variable 'b' is set.";
    }
  ?>
</body>
</html>
```

Return Value:	TRUE if <i>variable</i> exists and is not NULL, FALSE otherwise
Return Type:	Boolean
PHP Version:	4.0+
PHP Changelog:	PHP 5.4: Non-numeric offsets of strings now returns FALSE

The `isset()` function checks whether a variable is set, which means that it has to be declared and is not NULL. This function returns true if the variable exists and is not NULL, otherwise it returns false.

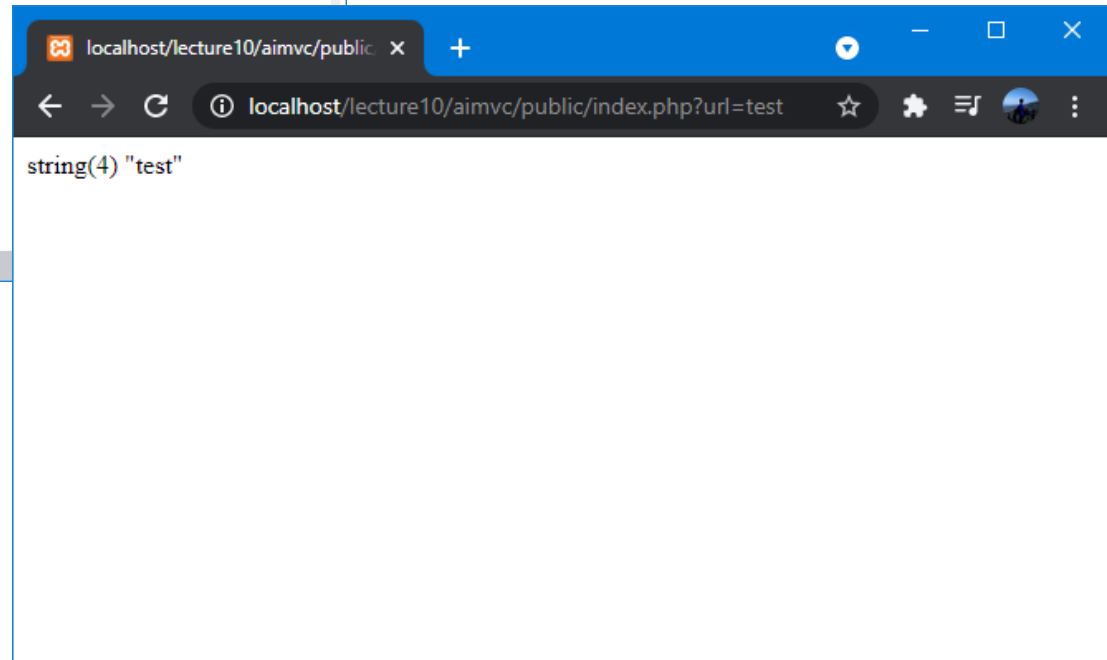
3 – Process Parsing URL



The screenshot shows the Sublime Text editor with the file `C:\xampp\htdocs\lecture10\aimvc\app\core\App.php` open. The left sidebar displays the project structure with folders like `aimvc`, `app`, `controller`, `core`, `model`, `view`, `public`, `css`, `img`, `js`, and `index.php`. The `App.php` file is selected in the `core` folder. The main editor window shows the following PHP code:

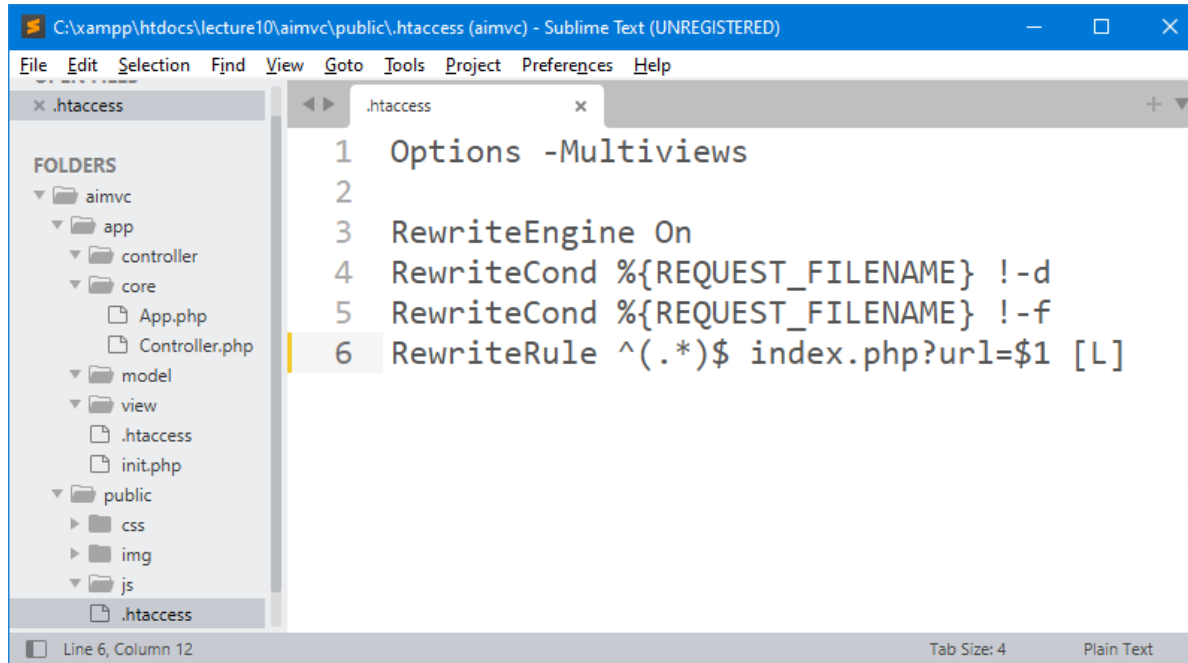
```
1 <?php
2 // Create a class
3 class App{
4     // PHP Constructor
5     public function __construct(){
6
7         // display structured information (type and value)
8         $url = $this->parseURL();
9         var_dump($url);
10
11     }
12
13     public function parseURL(){
14         if(isset($_GET['url'])){
15             $url = $_GET['url'];
16             return $url;
17         }
18     }
19 }
20
21 ?>
```

The status bar at the bottom indicates "Line 9, Column 26".



4 - Avoid Users Access (*public*)

Menghindari *kesalahan access file* atau **public** folder (*public directory*).



The screenshot shows a Sublime Text window titled "C:\xampp\htdocs\lecture10\aimvc\public\.htaccess (aimvc) - Sublime Text (UNREGISTERED)". The left sidebar displays a file explorer with the following structure:

- aimvc
 - app
 - controller
 - core
 - App.php
 - Controller.php
 - model
 - view
 - .htaccess
 - init.php
 - public
 - css
 - img
 - js
 - .htaccess

The main editor area shows the content of the .htaccess file:

```
1 Options -Multiviews
2
3 RewriteEngine On
4 RewriteCond %{REQUEST_FILENAME} !-d
5 RewriteCond %{REQUEST_FILENAME} !-f
6 RewriteRule ^(.*)$ index.php?url=$1 [L]
```

The status bar at the bottom indicates "Line 6, Column 12", "Tab Size: 4", and "Plain Text".

Create file **.htaccess** di dalam folder public. “**Options - Multiviews**” artinya kita akan menghindari kesalahan akses file ataupun directory dari URL (*Uniform Resource Locator*).

Karena *users* akan mengakses langsung *file* class bukan ke *folder* yang ada di **public directory** melalui URL. Maka kesalahan akses ataupun **SQL Injection techniques** (*backend database manipulation to access information*) dapat kita hindari.

4 - Avoid Users Access (*public*)

```
RewriteCond %{REQUEST_FILENAME} !-d  
RewriteCond %{REQUEST_FILENAME} !-f
```

Rewrite condition (**RewriteCond**) tests along with a corresponding rule (**RewriteRule**) if the prior conditions pass.

Jika URL yang di *request* adalah *directory/folder* (*d*) dan *file* (*f*) maka akan diabaikan (*ignored*). Dalam arti, *request* yang diinginkan hanyalah berupa *method*.

4 - Avoid Users Access (*public*)

RewriteRule **^(.*)\$ index.php?url=\$1 [L]**

RewriteRule defines a particular rule.

Fungsi Regex **^(.*)\$** artinya :

1. Symbol caret (^) artinya yang ditulis mulai dari awal url (*match the beginning of a line*) setelah penulisan url “/aimvc/public/”.
2. in regular expressions, symbol dot (.) is a special character used to match any one character. Jadi, akan ambil semua character sesudah “aimvc/public”.
3. A regular expression followed by an asterisk (*) matches zero or more occurrences of the regular expression. Jadi ambil character satu persatu dari url.
4. If a **dollar sign** (\$) is at the end of the entire **regular expression**, it matches the end of a line. Jadi akan dicek sampai semua character (*text*) yang ada dalam url selesai.

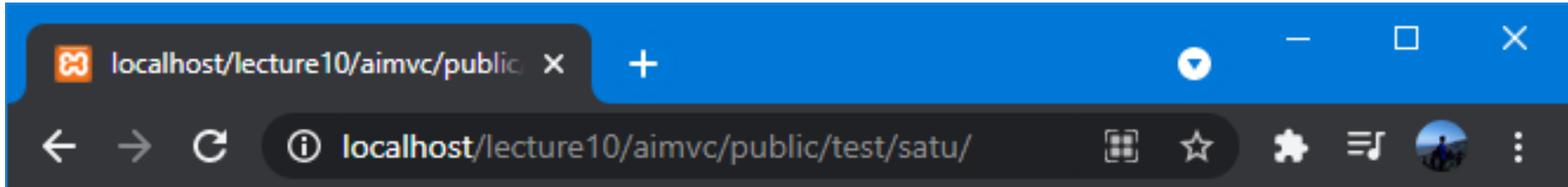
Fungsi Regex **index.php?url=\$1** artinya:

Arahkan ke *file* index.php yang mengirimkan url dan disimpan di \$1 (*placeholder*).

Flag **[L]** artinya:

- ☐ The [L] flag causes *mod_rewrite* to stop processing the rule set.
- ☐ In most contexts, this means that if the rule matches, *no further rules will be processed*.

Request URL *without* Format



string(10) "test/satu/"

Langkah awal untuk *Request URL, parsing URL* dan dapatkan controller class & method dengan memanfaatkan *.htaccess file* success !!!



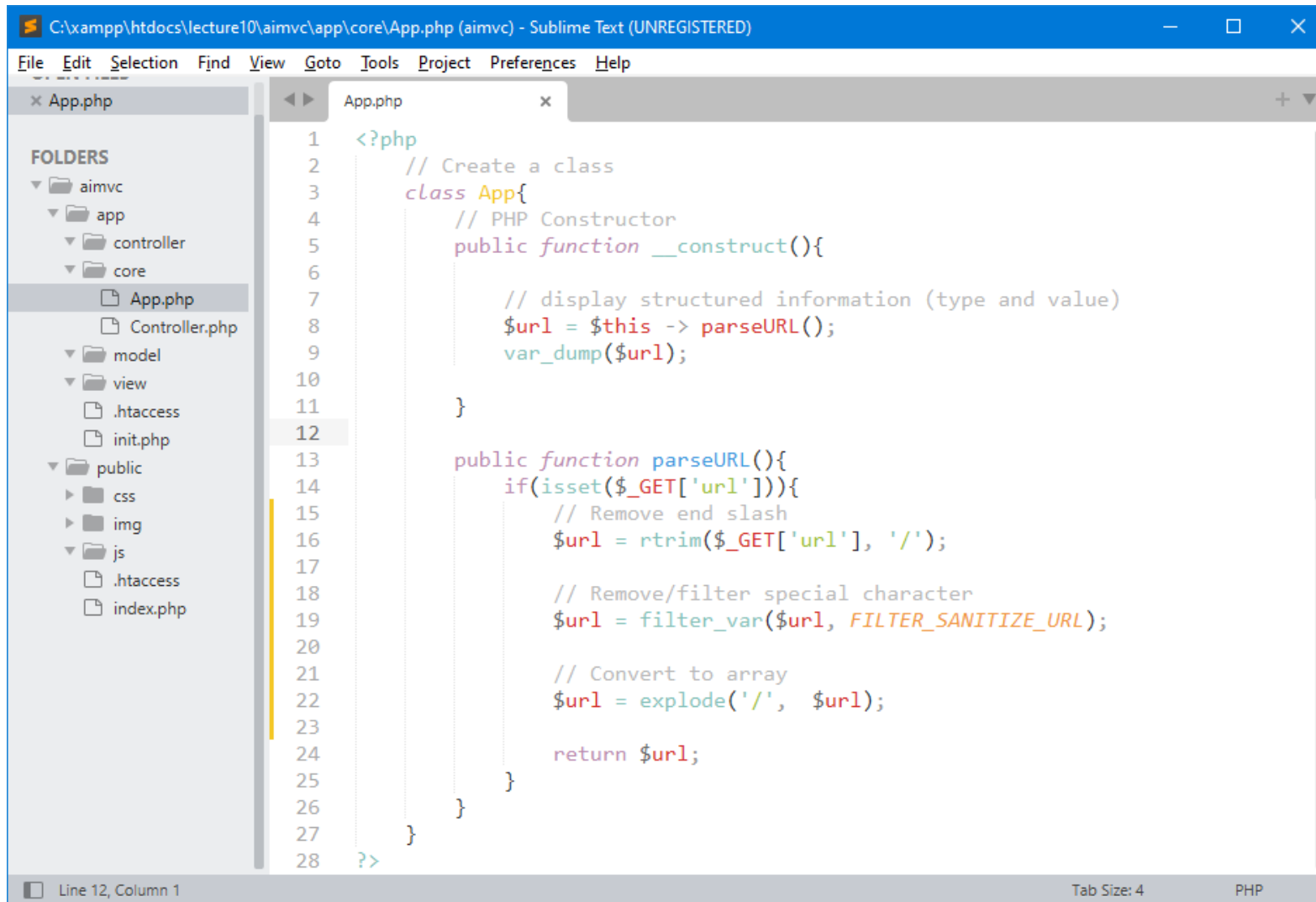
Setting *your* own routing rules

4 – Remove end “/” in Url, Special Character and Covert to Array.



Handling Request URL

Handle more *routing rules* when *users* try to request through URL.



```
1  <?php
2      // Create a class
3      class App{
4          // PHP Constructor
5          public function __construct(){
6
7              // display structured information (type and value)
8              $url = $this -> parseURL();
9              var_dump($url);
10
11         }
12
13     public function parseURL(){
14         if(isset($_GET['url'])){
15             // Remove end slash
16             $url = rtrim($_GET['url'], '/');
17
18             // Remove/filter special character
19             $url = filter_var($url, FILTER_SANITIZE_URL);
20
21             // Convert to array
22             $url = explode('/', $url);
23
24             return $url;
25         }
26     }
27 }
28 ?>
```

Test Router - Request URL





Exercise *for* students

Latihan untuk *Students*

Create directory : C:\xampp\htdocs\lecture11\

Create file : `_.php`

- ☐ Ikuti semua *steps* dan buat implementasikan dari *code* yang dibahas sebelumnya.
- ☐ Follow all the steps and make an implementation of the previously discussed codes.

END PRESENTATION

Thank you for your attention

Instructor: S – W – T

THANK YOU

